

## Input & Output

1. `print ("Hello", "World")`  
output : Hello World

2. `print ("=" * 30)`  
output : print "=" 30 times

3. `name = value`  
"=" sign assigns a value to a variable.

4. `type(x)`  
Returns the type of a value  
`str(text)`      `int(whole number)`      `float(decimal number)`

5. `Input("...")`  
It always return text string

6. `int(x)` : Convert to an integer  
`float(x)` : Converts to a decimal

7. `f"{x:2f}"` → displays number with 2 decimal places  
`f"{x:>10}"` → right aligns the value in a field with 10 character wide.

`137 // 60` → 2      (Quotient)  
`137 % 60` → 17      (Remainder)

(b) `> Input()` always give you the input data from the user and it is important to collect data

`> "20"` is the data stored in a string compared to 20 which is just a number

`> %` is the modulo operator which is used to find the remainder after dividing one number by another. It can be used for algorithm and mathematics tasks.

## W02

### 1. Comparisons

`=` [Equals to, which gives a true statement]

`!=` [Not equal to, gives a true statement]

`<` [is less than, gives a false statement]

`>` [is greater than, gives a true statement]

`<=` [is less or equal to, a false statement]

`>=` [is greater or equal to, a true statement]

`==`  $\rightarrow$  Is the equality operator / ask question

`=`  $\rightarrow$  is assignment operator to store a value in a variable | store data

### 2. Combining Conditions

and [logical AND, both statement has to be true to get true statement]

or [logical OR, at least one statement has to be true to get true statement]

not [logical NOT, statement is false]

### 3. Decisions

if [if statement, Booleans decide whether the statement is true]

elif [checked top to bottom. The first True statement runs and the rest skipped]

else [It runs when the 'if condition' was false]

### 4. Indentation

Intended lines are the block which needs to be respected in python. In loops for examples while using if statement.

### 5. While loops

It repeats the block while condition is True

It stops when condition becomes false (count  $\neq$  1)

`while True :` + `break`  $\rightarrow$  it loops until something inside the loop decides to stop instead of counting in advance.

`break` and `continue`  $\rightarrow$  the `break` exits the loop and `continue` skips the rest of the pass and goes straight to checking the condition.

## 6. For loops

`range (stop)` → when we know how many times to repeat

`range (3)` → starts from 0 to 3

`range (start, stop, step)` → Counting by more than one at a time, counting with a negative step.

`Enumerate ( )` → It hands the position and the value together

## 4. For or while

know the count in advance → For

waiting on a condition or a signal → while